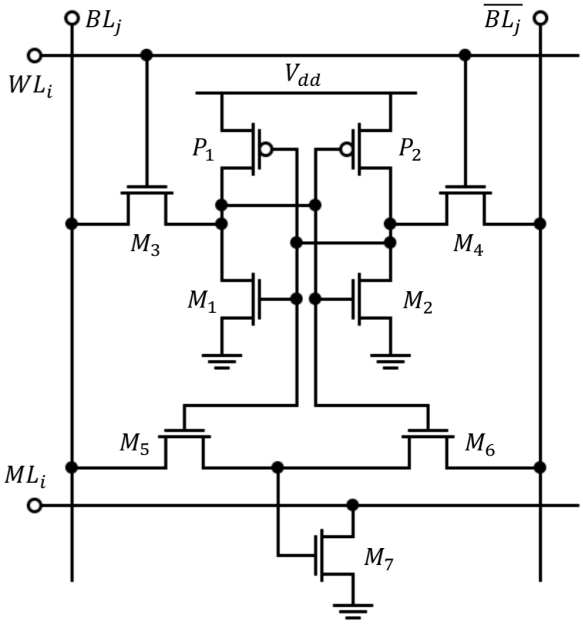
1 Lesson 13-10-26

1.1 Content addressable memories

CAMs (*Content Addressable Memories*), also known as *associative memories*, are special kinds of memory where the main operation is a **match operation**. Instead of providing the address and read the data, want to look for the *address* of some given data. This can be done in software, via indices or simple linear scans, but is far more efficient and faster (if not more expensive) to do in memory.

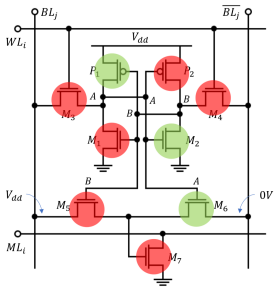
Let's look at the cell.



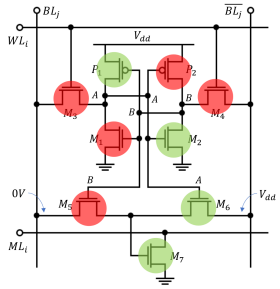
For the match operation, the word Line WL_i remains inactive for the entire operation: we don't want to change the bit cell content. Instead we recharge Match Line ML_i to V_{dd}, then apply the value to BL_i and its complement to $\overline{BL_i}$. If there is no match, ML_i will be discharged.

There are 4 main cases to study here:

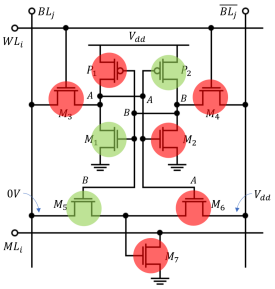
1. Contaning 1, matching 1:



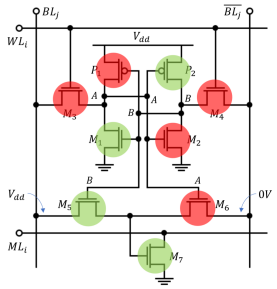
2. Contaning 1, matching 0:



3. Contaning 0, matching 0:

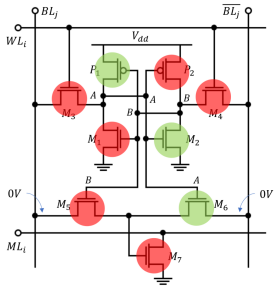


4. Contaning 0, matching 1:

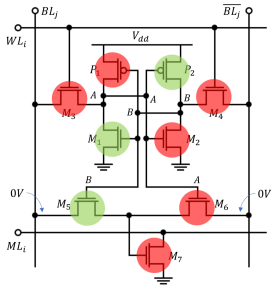


We then have other 2 cases, which relate to the *don't care* search state. We can perform a search by not caring if we set both bit lines to 0.

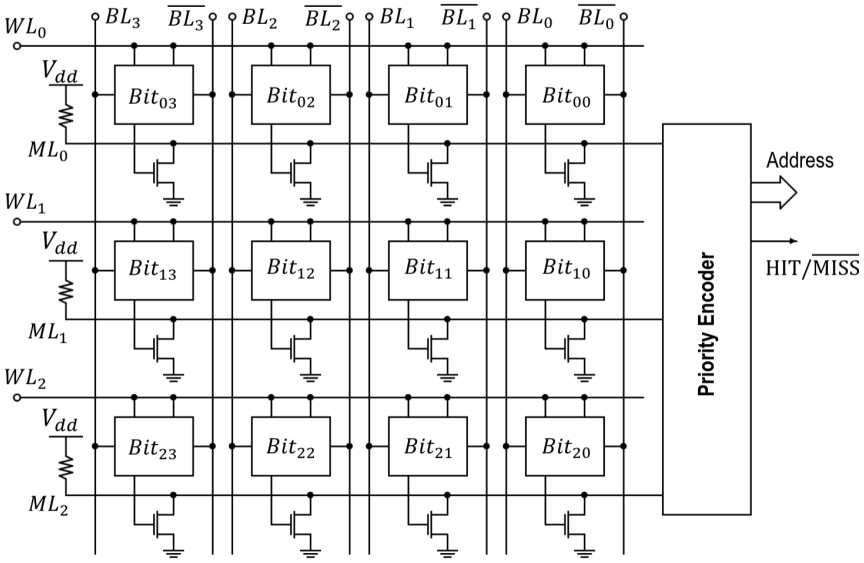
1. Contaning 1, don't care:



2. Contaning 0, don't care:



At this point we have documented the behavoir of the CAM cell enough to look at the actual topology:



Note that:

- Each Match Line spans an entire word, but not an entire row! This means that if all bits in Word i match, the Match Line ML_i is high. If on the other hand at least one bit in Word i does not match, the Match Line ML_i is low;
- Then note the *priority encoder*. If at least one Match Line is high, the priority encoder asserts HIT, and it generates the corresponding address (the highest). On the other hand, if all Match Lines are low, the priority encoder asserts MISS.

1.1.1 Associative SRAM

Let's look into how an actual **associative SRAM** is built from this premise.

In the simplest case an associative SRAM is only a circuit with the topology shown in the last section. When such an Associative SRAM is switched ON the content is random.

Let's assume some data is written, but only partially occupies the memory: we want to match only against the written data.

To do this, we have all words in the associative memory contain an additional bit, called the *valid bit*. At power on also the valid bit is also random, so a special associative memory operation initializes all valid bits to 0. This means marking all words as invalid

When writing a word, the corresponding valid bit is automatically set. At this point, when matching, all valid bits are automatically matched against logic 1, whereas all invalid words do not match.

1.1.2 Associative SRAM performance

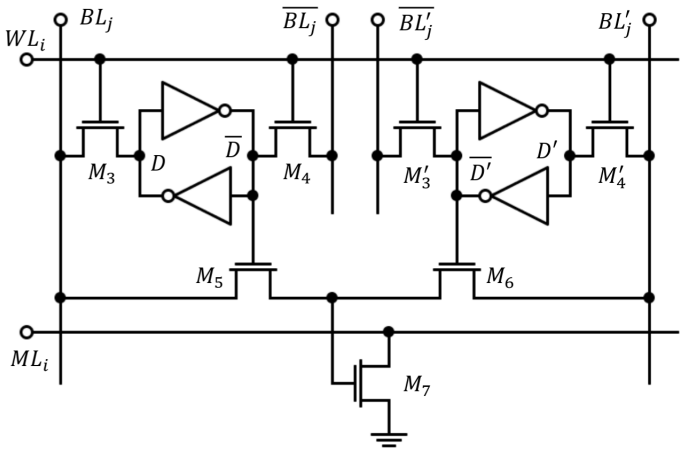
Doing matchin in hardware means all matching performed in parallel, which is much better than scanning the entire memory sequentially. This makes for a very quick result (tens of nanoseconds).

Of course, because of increased parasitic capacitances, the larger the memory, the slower the matching.

It goes without saying that such a ram is more expensive than an SRAM, with 9 transistors compared to 6 transistors for a single cell. The bit cell is also larger, has high power consumption, and especially high power consumption during the match operation (we have to precharge all match lines, only to discharge most (or all) of them).

1.1.3 Ternary CAM

There is another kind of CAM, which allows for *don't care* states to be encoded directly in the memory.



To do this we have two bit cells for each bit in the memory. We map the usual `10` and `01` values to logic 1 and 0 (same as normal SRAM), but also reserve `11` for don't care.

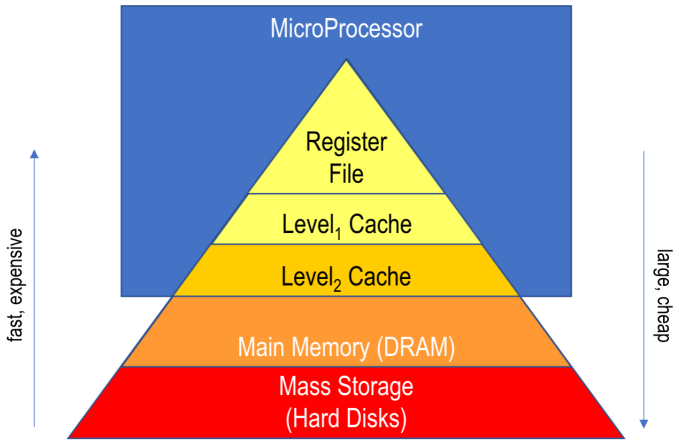
Matching can be done the same, as searching for `00` when the actual value is `11` keeps the ML_i high.

This kind of memory is really useful in *routers*, where routing tables often involve bits which are not specified by the address mask. For *cache* memories, on the other hand (which will be the devices we will study later), we don't care about this functionality, and as such ternary CAMs are *not used*(!).

1.2 Cache memory

Computing systems need large, fast and cheap memories. We saw a few options:

- **DRAM**: large, cheap but not fast;
- **SRAM**: fast, but not large and cheap;
- **Flip-flops**: we can implement *registers* with these which are as fast as we can get.



At this point we note that if SRAM is small, it becomes cheap, so there is some value in building a hierarchy of different memories to get high performance at low cost.

- Expensive (and fast) memories should be used to store little, frequently accessed data. An SRAM is a great fit for this;
- Cheap (and slow) memories should be used to store the bulk of the data. DRAMs is also a great fit for this.

Note that the computing system should not be aware of the differences: it simply sees a large, fast and cheap memory.

A clear problem is that data in the large DRAM does not fit into the small SRAM. As we said, we intend to keep fractions of the DRAM in cache. To maximize the use of the cache, we should ideally predict the data that is required next, and transfer it from DRAM in cache before it is needed.

1.2.1 Hits and misses

We refer by **MISS** to the situation where data is queried but not present in cache, and by **HIT** to the situation where data is queried and indeed present in cache.

- For a MISS we have to check both the cache and the DRAM for actual data, so the total time the read operation takes is:

$$t_{\text{MISS}} = t_{\text{cache}} + t_{\text{DRAM}}$$

- For a HIT we have to check only the cache, so the total time the read operation takes is:

$$t_{\text{HIT}} = t_{\text{cache}}$$

  By defining the hit rate H as:

$$H = \frac{n_{\text{HIT}}}{n_{\text{TOT}}}$$

  We get that the average time is:

$$t_{\text{avg}} = Ht_{\text{HIT}} + (1-H)t_{\text{MISS}} = Ht_{\text{cache}} + (1-H)(t_{\text{cache}} + t_{\text{DRAM}})$$

  Clearly, higher H is better.

1.2.2 Predictions

To (somewhat) accurately predict the data that the microprocessor will need we exploit **locality** of data. There are 2 types of locality:

- **Temporal** locality: the same data is used more often if it was recently referenced;
- **Spatial** locality: data around the data which was just used is more likely to be used next.

Instructions, for instnace, fit exactly in this framework as they are highly local (if we fetch instruction at `0x12` we will likely fetch at `0x16` next, assuming 32 bit instructions).

1.2.3 Cache performance

Let's talk about performance. A large cache potentially makes execution faster, and if more data fits in the cache, the HIT Rate is higher. Once all code and data is in cache we see no improvement in making the cache even larger.

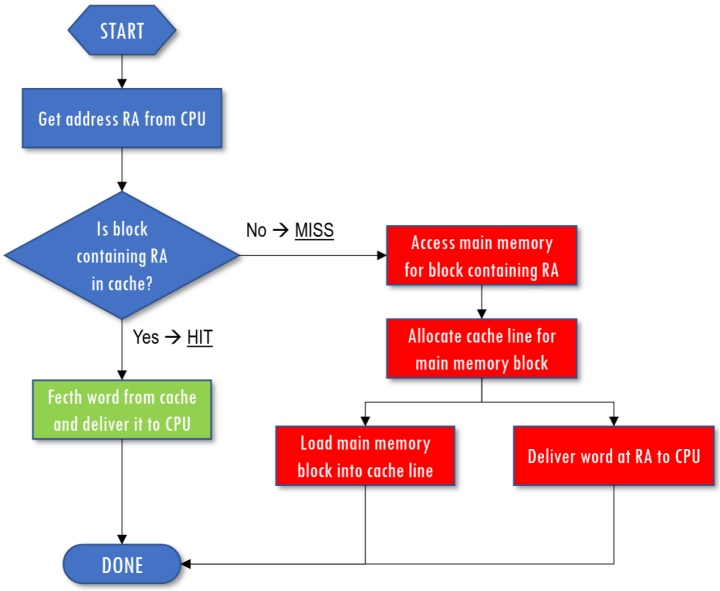
Also, large memories are slower, with higher parasitic capacitances (meaning higher t_{cache}). Checking cache for data takes time, and large memories consume more power.

A large cache is also more expensive, and the cost is additional to the large DRAM.

This makes cache sizing a trade-off between costs and performance, which is often application dependent.

1.2.4 Cache policy

Let's look at how actual cache systems are structured.



First we introduce the idea of a minimum unit of storage in cache, that is the **cache line**. Data in DRAM is divided in **blocks**, with each block the same size of a cache line. Blocks are aligned to the cache line size, and data is always moved to and from the cache in blocks.

At this point the microprocessor generates a main memory address, called **RA** (*Read Address*). Of course, he word at RA belongs to a certain block.

If that word is in cache (that is, in a certain cacheline), we have to note addresses in DRAM and cache are different. This means we need to find a fast way of knowing whether a given main memory block is alread stored in cache or not, and where.

The required data is identified by its address RA in main memory. Since data is transferred between main memory and cache in blocks, we can identify each block using its block address BA, obtained from RA. To determine whether a block is already present in the cache, we can use a *Content-Addressable Memory* (CAM), in addition to the SRAM used to store the actual data. The CAM contains one entry per cache line, storing the corresponding main memory block address BA, or just enough of it to identify the block. When a memory access occurs, the processor computes BA from RA and searches for a matching entry in the CAM. If a match is found, we have a *cache hit*: the CAM identifies the corresponding cache line, allowing the requested word to be retrieved from SRAM. Otherwise, we have a *cache miss*: the block is fetched from DRAM, stored in a cache line, and its address is recorded in the CAM. The requested word can then be returned to the processor.

This is usually referred to as *fully associative* caches. We'll see that it's not the only approach.